Student ID: Student Name:

Course: Data Structures (CSE CS203A)

Assignment V: Tree

Due date: 2025.12.30 23:59:59

Important Notice – Use of AI Tools

In this assignment, you must use at least one AI assistant (e.g. ChatGPT, Gemini, Claude, Grok, M365 Copilot) as a learning tool to help you:

- review definitions,
- compare tree variants, and
- organize your report.

You are not allowed to let the AI directly produce your final diagrams or final report content without your own understanding and rewriting.

You must log all AI prompts and services used (see "AI Usage Log" section below).

1. Goal of This Assignment

In the lectures, we introduced the concept of the tree as a data structure, starting from the general tree and then moving to more specialized forms.

In this assignment, you will:

a. Understand and clearly define:
   1. General tree
   2. Binary tree
   3. Complete binary tree
   4. Binary search tree (BST)
   5. AVL tree
   6. Red-Black tree
   7. Max heap
   8. Min heap

b. Build a hierarchy and transformation path from the general tree to these variants, and explain how each variant adds more structure or constraints.

c. Use a fixed list of integers to construct multiple tree variants and visualize them.

d. Choose one real-world application for each tree type and explain why that data structure fits the application.

e. Practice using AI tools as study companions and keep a simple Q&A log.

2. Given Data

Use the following 20 integers as the input data for all your tree constructions:

37, 142, 5, 89, 63, 117, 24, 176, 58, 133, 92, 11, 151, 72, 39, 184, 7, 101, 54, 160

Student ID: Student Name:

You will reuse this same sequence for every tree type (binary tree, complete binary tree, BST, AVL, Red-Black, max heap, min heap).

3. Deliverables

Please complete your work in the Student Worksheet Companion and upload it to the YZU Portal System.

Your report should include the following parts:

a. Definitions (Concept Review)

Provide clear, concise definitions for each of the following:

1. General tree
2. Binary tree
3. Complete binary tree
4. Binary search tree (BST)
5. AVL tree
6. Red-Black tree
7. Max heap
8. Min heap

You are encouraged to use AI tools to help you understand these concepts, but you must rewrite the definitions in your own words.

b. Hierarchy and Transformation of Tree Variants

Based on the definitions above, build a "tree family hierarchy" that shows how these structures are related. For example:

- General tree → Binary tree
- Binary tree → Complete binary tree / Binary search tree
- BST → AVL tree / Red-Black tree
- Binary tree → Max heap / Min heap

Tasks:

- Draw a diagram or flow chart that shows the transformation or specialization path: general tree → binary tree → complete binary tree → BST → AVL/Red-Black, etc.
- For each arrow (transformation), briefly explain:
  - What new constraint or property is added?
  - e.g., "Binary tree = tree with at most 2 children per node",
    "BST = binary tree with left < root < right",
    "AVL = BST with strict height-balance rule", etc.

c. Tree Construction with the Given Integers

Using the given 20 integers, construct the following tree variants:

1. Binary tree
2. Complete binary tree
3. Binary search tree (BST)

4. AVL tree
5. Red-Black tree
6. Max heap
7. Min heap

Important Hint / Restriction:

- For these trees, you must use tree visualization tools (e.g., online visualizers or software) to build and display the tree.
- You may not ask AI tools to directly generate the final tree pictures for you.
- Instead:
    - Use AI only to help you understand algorithms,
    - Then apply those algorithms in a visualizer (or your own implementation).

What to submit for this part:

For each tree type:

- A snapshot (image) of the constructed tree.
- The URL / name of the visualization tool you used.
- A short note on how you inserted the integers (e.g., "insert in the given order as BST", "build max heap using heapify", etc.).

d. Application Example for Each Tree

For each of the following:

1. Binary tree
2. Complete binary tree
3. Binary search tree (BST)
4. AVL tree
5. Red-Black tree
6. Max heap
7. Min heap

Choose one application (real-world or system-level) and explain:

1. Application description
    - e.g., priority scheduling, dictionary lookup, memory allocation, database indexing, etc.
2. Why this tree structure fits
    - What property of this data structure makes it suitable?
    - Example:
        - Max heap → good for priority queue because the largest element is always at the root, so extracting max is efficient.
        - Red-Black tree → good for standard library maps/sets because it guarantees O(log n) operations even under many insertions/deletions.

Your explanation should show that you understand the link between the data structure and its use case.

e. Report Layout and Organization

You are free to design the layout of your report, but it should:

- Be well-structured (use sections, headings, tables, and diagrams).
- Have a clear flow from:
  - definitions →
  - hierarchy/transformation →
  - constructed trees →
  - applications →
  - AI usage log.
- Be easy for another student to read and learn from.

Feel free to use AI to suggest a good outline, but you must decide and finalize the layout yourself.

f. AI Usage Log (Q&A Table)

Every time you use an AI copilot service for this assignment, record:

- Index (1, 2, 3, …)
- Prompt (what you asked)
- Service (e.g., ChatGPT, Gemini, Copilot, …)

Example log table:

Index	Prompt	Service
1	Assist me to have the definition of general tree, binary tree, complete binary tree, binary search tree, AVL tree, red-black tree, max heap and min heap for self-learning.	ChatGPT
2	Explain the difference between AVL tree and Red-Black tree in terms of balancing strategy and use cases.	Gemini
…	…	…

Place this table at the end of your report.

4. Evaluation (100 pts)

A possible breakdown (you can adjust if needed):

a. Concept definitions (20 pts)

- Correctness and clarity of all 8 tree type definitions.

b. Hierarchy & transformation explanation (20 pts)

- Clear diagram / explanation of how each tree variant evolves from the general tree.
- Correct identification of constraints/invariants.

c. Tree constructions & visualizations (25 pts)

- Correct constructions for each tree type using the given integers.
- Proper screenshots and tool URLs.

- Consistent insertion / heap-building strategy descriptions.

d. Applications & explanations (20 pts)
   - One application per tree type.
   - Clear explanation linking data structure properties to the application.

e. Report organization & AI usage log (15 pts)
   - Logical report structure and readability.
   - AI log completeness (all prompts listed with service names).
   - Thoughtful use of AI as a learning assistant, not as a copy-paste generator.

Student ID: Student Name:

Student ID: 1133320 Student Name: 曾偉翔

Course: Data Structures (CSE CS203A)

Assignment V: Tree

Student Worksheet Companion

Due date: 2025.12.30 23:59:59

Academic Integrity and AI Usage Statement

In this assignment, you must use AI tools (such as ChatGPT, Gemini, Claude, Grok, M365 Copilot, etc.) as learning assistants, but you must also take full responsibility for understanding and organizing your own work.

1. Permitted Use of AI Tools

   You may use AI to:

   - Review or clarify definitions and concepts.
   - Compare different tree data structures.
   - Get suggestions for report layout or examples.
   - Ask for explanations of algorithms (e.g., BST insertion, AVL rotation, heapify process).

   You should read, think about, and rewrite the content in your own words.

2. Not Permitted
   - Do not copy/paste AI-generated content directly as your final answer.
   - Do not ask AI to draw the final diagrams or directly produce the final tree screenshots.
   - Do not ask AI to complete the whole assignment report for you.
3. Your Responsibility
   - You are responsible for understanding the definitions and algorithms.
   - You are responsible for verifying whether AI answers are correct or not.
   - You must produce your own original explanations and diagrams.
4. AI Usage Log
   - You must record all AI queries related to this assignment.
   - At the end of your report, include an AI Usage Log table with: Index, Prompt, AI service name.

   By submitting this assignment, you acknowledge that you have used AI tools only as study aids, and that the final content of this assignment represents your own understanding and work.

Section 1. Definitions of Tree Variants

Task: Write your own definitions for each tree type. You may use AI for learning, but rewrite in your own words.

Student ID: Student Name:

1. General Tree
   Definition:每個節點的子節點可以有任意數量個
2. Binary Tree
   Definition:每個節點的子節點最多只能有兩個

3. Complete Binary Tree
   Definition:承襲 Binary Tree 結構，但額外規定除最後一層外所有層都要填滿，最後一層必須從左至右填入

4. Binary Search Tree (BST)
   Definition:承襲 Binary Tree 結構，但有額外規定此樹要滿足左子節點<父節點<右子節點，用於高效率查找

5. AVL Tree
   Definition:承襲 BST 結構(A self-balancing BST)，但額外規定左右子樹的高度差不超過 1

6. Red-Black Tree
   Definition:承襲 BST 結構(A self-balancing BST)，節點標記為紅或黑，透過顏色規則與旋轉保持平衡

7. Max Heap
   Definition:承襲 Complete Binary Tree 結構，每個父節點≥子節點，最大值在根

8. Min Heap
   Definition:承襲 Complete Binary Tree 結構，給個子節點≥父節點，最小值在根

Section 2. Tree Family Hierarchy and Transformations

Task: Show how these structures are related (general → specialized). Use a simple diagram and explanations of what constraints are added at each step.

2.1 Tree Family Diagram

You may draw this by hand and paste a photo, or use drawing tools.

Suggested chain example (you may extend or adjust):

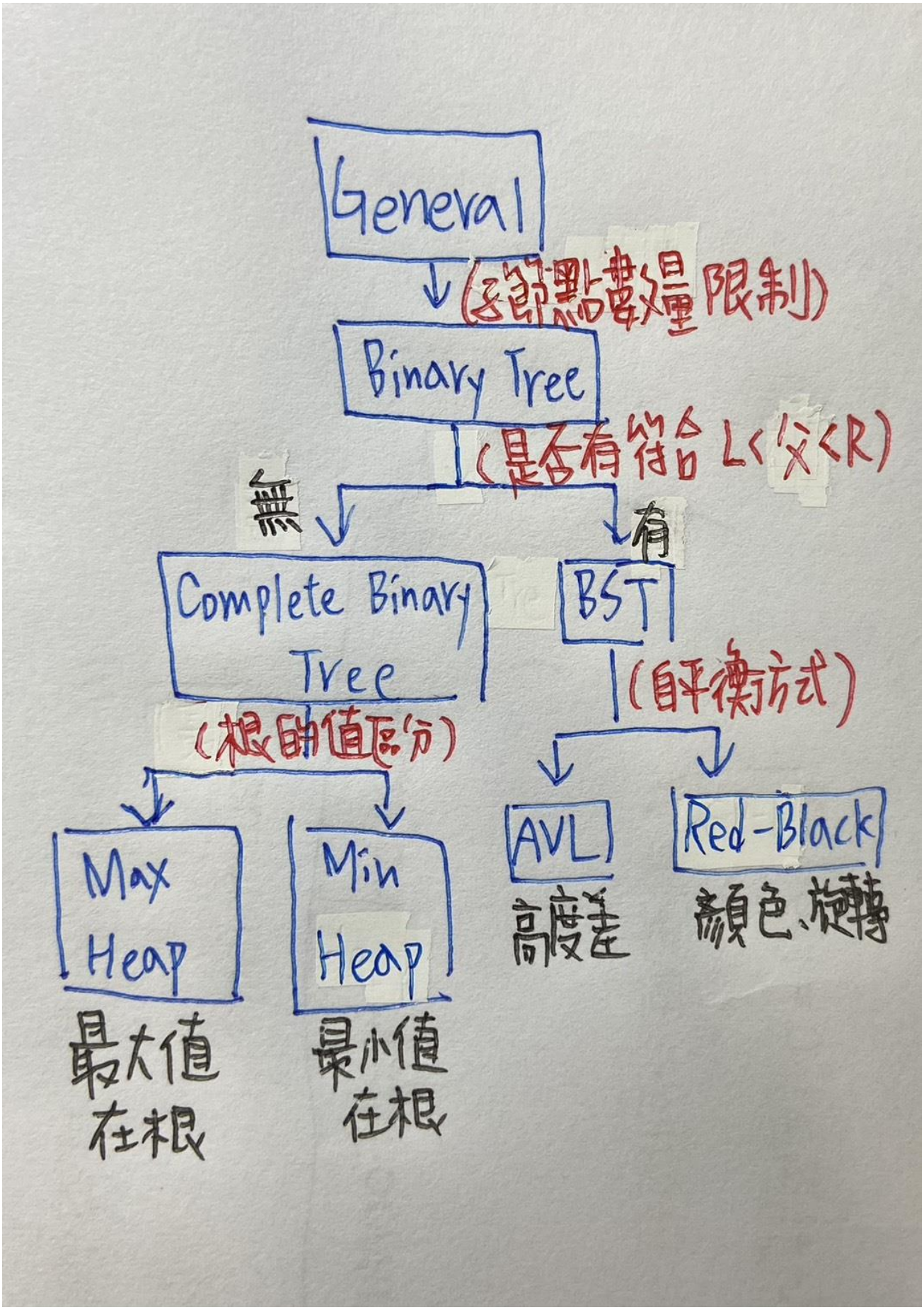
General Tree → Binary Tree → Complete Binary Tree

Binary Tree → Binary Search Tree → AVL / Red-Black

Binary Tree → Max Heap / Min Heap

Your Diagram:

Student ID: Student Name:



Student ID: Student Name:

2.2 Explanation of Transformations

Fill in what new property or constraint is added at each step.

From	To	New property / constraint added
General Tree	Binary Tree	每個節點的子節點最多只能 2 個
Binary Tree	Complete Binary Tree	除了最後層外都要填滿，最後層節點從左至右填入
Binary Tree	Binary Search Tree	排序限制:左子<父節點<右子
BST	AVL Tree	高度平衡限制:左右子樹高度差≤1
BST	Red-Black Tree	顏色平衡規則:節點標記為紅或黑，遵守旋轉與顏色規則
Binary Tree	Max Heap	堆積順序約束；父節點≥子節點，根節點為最大值
Binary Tree	Min Heap	堆積順序約束；子節點≥父節點，根節點為最小值

Section 3. Tree Constructions Using Given Integers

Given integers (fixed for all parts):

37, 142, 5, 89, 63, 117, 24, 176, 58, 133, 92, 11, 151, 72, 39, 184, 7, 101, 54, 160

Task: For each tree type below, construct the tree using these integers, take a screenshot of the tree from your chosen tool, record the tool name/URL, and describe the insertion / heap-building procedure.

3.1 Binary Tree

Tool name / URL:

Binary Tree Visualizer and Converter — TreeConverter

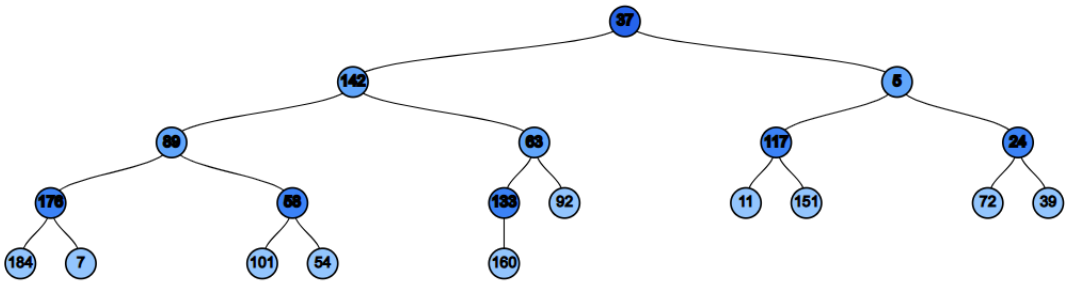
URL: Binary Tree Visualizer and Converter

Construction / insertion description:

因為 Binary Tree 的限制只有每個節點最多只有兩個子節點，因此插入數字時只要確保子節點數量的部分正確即可

Screenshot of Binary Tree (paste below):

Student ID: Student Name:



3.2 Complete Binary Tree

Tool name / URL:

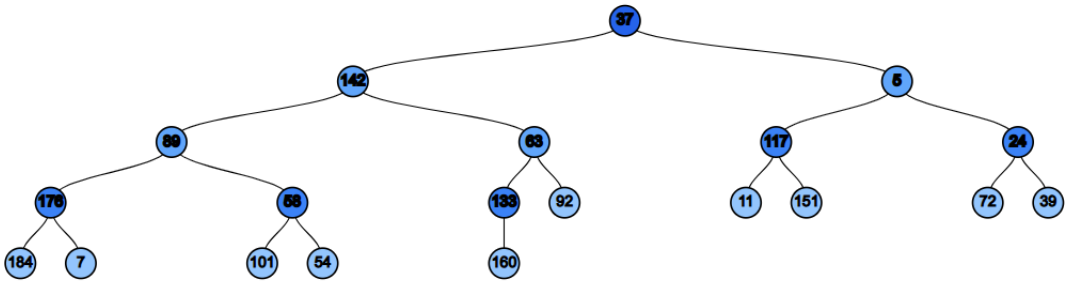
Binary Tree Visualizer and Converter — TreeConverter

URL: Binary Tree Visualizer and Converter

Construction / insertion description:

依照題目給的數字順序依序填入。Complete Binary Tree 是承襲 Binary Tree，但要注意除最後一層外都要填滿，最後一層要從左至右填入

Screenshot of Complete Binary Tree (paste below):



3.3 Binary Search Tree (BST)

Tool name / URL:

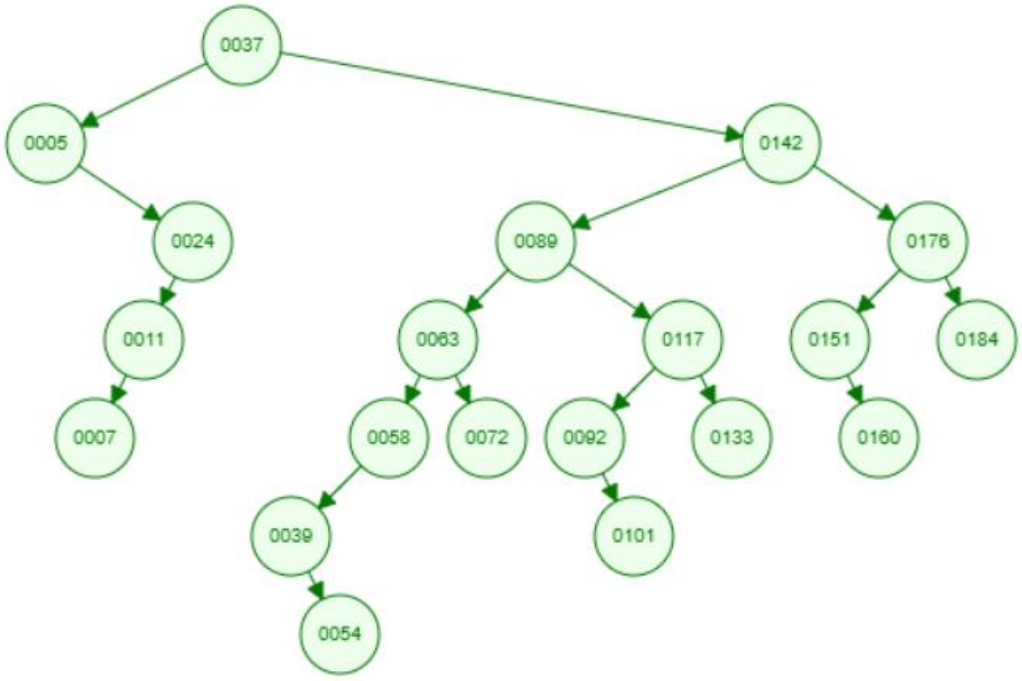
Student ID: Student Name:

Binary Search Tree Visualization (University of San Francisco)

URL:Binary Search Tree Visualization

Insertion rule (e.g., “insert in given order using BST rules”):
依照題目給的數字順序依序填入，且遵守 BST 規則: 左子<父節點<右子

Screenshot of BST (paste below):



3.4 AVL Tree

Tool name / URL:

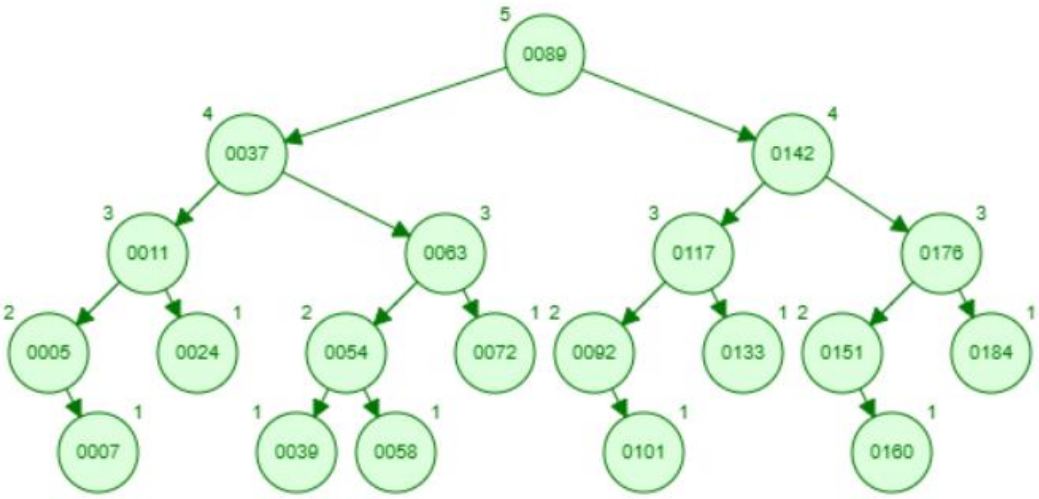
USF Data Structure Visualizations

AVL Tree Visualzation

Insertion & balancing description:
將整數序列依序插入到 AVL Tree，遵循 BST 規則，且要更新高度，每次插入後更新沿途節點的高度與平衡因子，若失衡則透過旋轉修正

Screenshot of AVL Tree (paste below):

Student ID: Student Name:



3.5 Red-Black Tree

Tool name / URL:

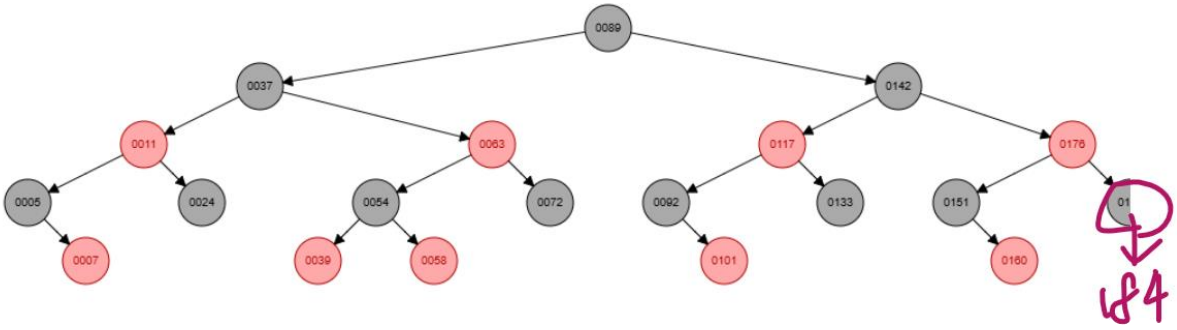
USF Red-Black Tree Visualization

Red/Black Tree Visualization

Insertion & balancing description:

將整數序列依序插入到 Red-Black Tree，遵循 BST 規則，插入的新節點一律標記為紅色，根節點、所有葉子(NIL)節點必須是黑色，紅色節點不能有紅色子節點，從任一節點到其所有葉子的路徑，黑色節點數必須相同，每個節點要麼是紅色要麼是黑色

Screenshot of Red-Black Tree (paste below):



3.6 Max Heap

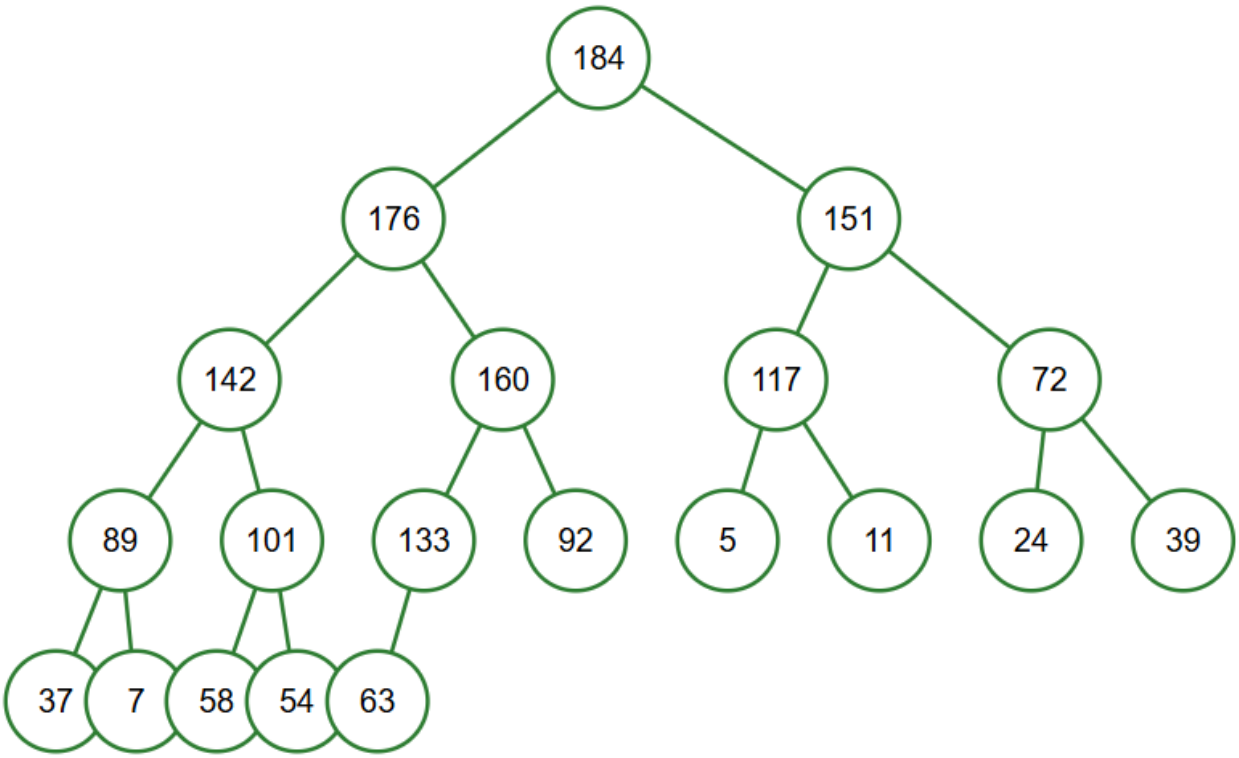
Tool name / URL:

MAX Heap Visualization

https://sercankulcu.github.io/files/data_structures/slides/Bolum_08_Heap.html

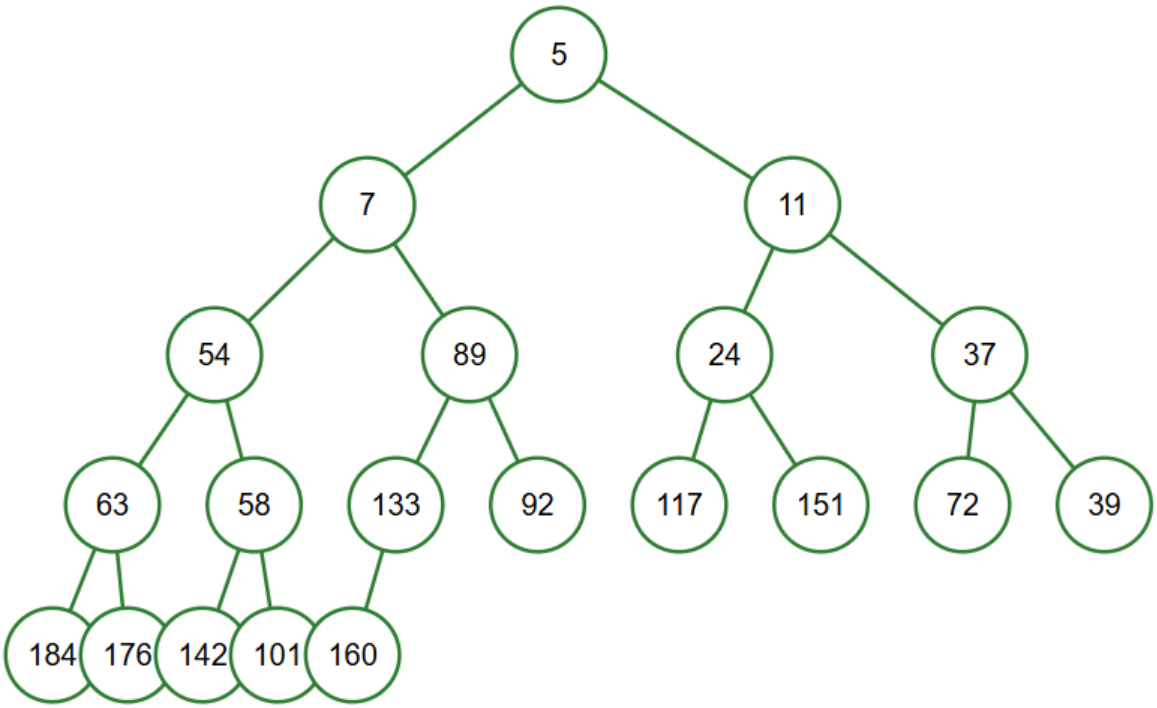
Student ID: Student Name:

Construction / heap-building description (e.g. heapify, insert-and-sift-up):
1.依序插入整數
2.檢查堆續性:每次插入後，若新節點>父節點，執行上浮
3.交換過程:新節點與父節點交換，直到父>子，或到達根節點
4.每次插入後，樹仍保持 Complete Binary Tree 結構，且父節點≥子節點
Screenshot of Max Heap (paste below):



3.7 Min Heap
Tool name / URL:
MIN Heap Visualization
[Heap Simulator - Interactive Min Heap Visualization]
Construction / heap-building description:
1.依序插入
2.檢查堆續性:　每次插入後，若新節點 < 父節點，則執行上浮
3.交換過程:新節點與父節點交換，直到父≤子，或到達根節點
4.每次插入後，樹仍保持 Complete Binary Tree 結構，且父節點≤子節點
Screenshot of Min Heap (paste below):

Student ID: Student Name:



Section 4. Application Examples

Task: For each tree type, choose one application and explain why this tree is suitable.

Tree Type	Application Example (name / context)	Why this tree fits (properties that matter)
Binary Tree	Expression Tree (用於編譯器)	每個運算符最多有兩個操作數，Binary Tree 結構自然對應數學表達式
Complete Binary Tree	Array-based representation (陣列儲存結構)	Complete Binary Tree 能緊密映射到陣列索引，節省空間、方便存取
Binary Search Tree	Database indexing (資料庫索引)	BST 保持有序，支援快速查找、插入、刪除，平均時間複雜度 O(log n)
AVL Tree	Memory allocation system (記憶體管理)	AVL 樹高度嚴格平衡，查找效率穩定，適合需要頻繁查找的場景
Red-Black Tree	STL map/set (C++標準模板庫)	插入/刪除效率高，旋轉次數少，保證 O(log n)操作，適合通用容器
Max Heap	排程系統(Job scheduling, priority queue)	根節點永遠是最大值，能快速取得最高優先級任務
Min Heap	Dijkstra 演算法 (最短路徑)	根節點永遠是最小值，能快速取得當前最短距離節點

Section 5. Reflection on Tree Family and Performance (Optional but recommended)

Student ID: Student Name:

Among BST, AVL, and Red-Black trees, which one would you pick for:
Mostly search (few updates)? Why?
答案:AVL Tree
原因:AVL 樹高度嚴格平衡，查找效率最穩定，時間複雜度接近 O(log n)
因為更新操作少，所以旋轉開銷不大，能充分發揮 AVL 在查找上的優勢
適合查找的場景，例如:資料庫索引、快取

Frequent insertions and deletions? Why?
答案:Red-Black Tree
原因:雖然查找效率低於 AVL(因平衡較鬆散)，但插入和刪除時旋轉次數更少
保證高度在 O(log n)，同時維持較低的維護成本
適合更新頻繁的場景，例如 STL map/set、作業系統的排程器

If you must store these 20 integers for static search only (no updates), which structure or representation would you prefer (sorted array + binary search, BST, AVL, etc.)? Why?
答案: sorted array + binary search
原因:
1. 資料不會更新，因此不需要樹的插入/刪除功能
2. 陣列比樹更簡單，空間利用率高，存取快
3. Binary Search 時間複雜度 O(log n) 且不需要旋轉或維護樹結構
4. 陣列的特性:固定大小，元素有序排列

Section 6. AI Usage Log (Required)

Task: Record every time you ask an AI assistant about this assignment.

Index	Date / Time	AI Service (ChatGPT, Gemini, etc.)	Your Full Prompt / Question
1	12/13 13:01	Copilot	請告訴我這些資料結構:General tree 、Binary tree 、Complete binary tree 、Binary search tree (BST) 、AVL tree 、Red-Black tree 、Max heap 、Min heap 的定義，並比較之間的差異，讓我更清楚它們的結構
2	12/13 15:21	Copilot	紅黑樹有高度差限制嗎?
3	12/13 15:51	Copilot	有哪些工具可以讓 AVL Tree 可視化?
4	12/13 16:02	Copilot	有哪些工具可以讓 Red-Black Tree 可視化?
5	12/13 16:25	Copilot	有哪些工具可以讓 Max/Min Heap 可視化?

Student ID: Student Name:

6	12/13 16:49	Copilot	請幫我整理每種樹的應用表格
7	12/13 17:11	Copilot	請告訴我什麼是 Dijkstra 演算法
8	12/13 17:30	Copilot	請整理 Red-Black Tree 和 AVL tree 在查找/插入/刪除上的差異

You may extend this table as needed.